

ISW - Tutorato

s.dessi63@studenti.unica.it

l.piras78@studenti.unica.it

OGGETTO MAIL "ISW"

Cosa vedremo oggi

- ▶ Moduli
- ▶ Package
- ▶ Esempi
- ▶ Lambda Functions
- ▶ RegEx
- ▶ Debugger

Moduli

- ▶ Un modulo Python è un file contenente definizioni di funzioni, classi e variabili che possono essere utilizzati all'interno di altri file di codice
- ▶ I moduli sono spesso utilizzati per organizzare il codice in unità logiche, come funzioni correlate o classi che forniscono un'astrazione di un concetto o di un dato.
- ▶ Le informazioni contenute in un certo modulo possono essere importate in un altro modulo.

Package

- ▶ Un "package" è un modo di organizzare e strutturare una collezione di moduli correlati.
- ▶ È essenzialmente una directory contenente un file speciale chiamato `__init__.py`, che può essere vuoto o può contenere codice di inizializzazione per il package.
- ▶ Questo file dice a Python che la directory dovrebbe essere trattata come un package.
- ▶ Un package può contenere sotto-package o moduli.

Percorso di ricerca di un modulo

- ▶ Quando un modulo chiamato ad esempio script è importato, l'interprete Python prima di tutto cerca un modulo built-in con quel nome.
- ▶ Nel caso in cui non lo trovasse cerca un file chiamato script.py in una lista di percorsi data dalla variabile `sys.path`. Questa variabile è inizializzata nelle seguenti posizioni:
 - ▶ La directory contenente lo script che si sta scrivendo;
 - ▶ `PythonPath` (Il percorso di ricerca viene esteso a tutte quelle directory che hanno a che fare con python);
 - ▶ Il default di installazione.

Moduli & Package

Per importare un modulo si utilizza l'istruzione **"import"** o **"from x import"** :

import nome_modulo **from** nome_modulo **import** ...

- ▶ Con questa istruzione viene solamente importata la referenza del modulo di nome nome_modulo.
- ▶ Se nel file nome_modulo.py viene definita una variabile foo = 10 questa sarà utilizzabile usando come prefisso il nome del modulo nome_modulo.foo.
- ▶ La sintassi "from...import" consente invece di importare solo alcune funzioni specifiche di un modulo, rendendole disponibili senza il prefisso del nome del modulo.
- ▶ può rendere il codice più leggibile e ridurre la possibilità di conflitti di nomi (ovvero la situazione in cui due definizioni con lo stesso nome vengono importate da moduli diversi)

Importare un modulo

```
/dir_progetto  
|  
|---- modulo_1.py  
|  
|---- modulo_2.py  
|  
|---- main.py
```

```
modulo_1.py def  
somma(a, b):  
return a + b
```

```
modulo_2.py  
def sub(a, b):  
return a - b
```

```
main.py
```

```
import modulo_1 import modulo_2
```

```
Print( modulo_1.somma(1,1), modulo_2.sub(1,1))
```

Moduli & Package: esempio moduli

Per evitare di utilizzare il prefisso del modulo è possibile assegnare l'import di una definizione ad una variabile.

```
import modulo_1
```

```
somma = modulo_1.somma
```

```
print(somma(1,2),2) )
```

```
import modulo_1 as m
```

```
somma = m.somma
```

```
print(somma(1,2),2))
```

- ▶ Quando si importa un modulo, l'interprete python esegue tutte le istruzioni in esso contenute.
- ▶ In questo modo è possibile inizializzare il modulo stesso quando questo viene importato.
- ▶ Le eventuali istruzioni vengono eseguite soltanto la prima volta che il modulo viene importato.

Importare un Package

```
MyPackage
|
|---- init.py
|
|---- modulo_1.py
|
|---- modulo_2.py
```

```
modulo_1.py def
somma(a, b):
return a + b
```

```
modulo_2.py
def sub(a, b):
return a - b
```

```
import MyPackage.modulo1
from MyPackage import modulo2
```

```
#from MyPackage.modulo2 import sub
```

```
Print( MyPackage.modulo1.somma(1,1),
modulo_2.sub(1,1))
```

Moduli & Package: esempio moduli

- ▶ Per importare un modulo o una funzione da un package si utilizza l'istruzione "import" o "from x import" :
- ▶ La differenza con i moduli sta nel utilizzo del punto "." per indicare la gerarchia delle cartelle, per esempio:
- ▶ Import MyPackage.modulo1 indica che "MyPackage" è il package principale mentre "modulo1" può essere un modulo o un sotto-package.
- ▶ Import MyPackage.modulo1.modulo2 indica che "MyPackage" è il package principale mentre "modulo1" deve essere un sotto-package o un modulo
 - ▶ modulo2 può essere sia una funzione(se modulo1 è un modulo) o un modulo.

Mathematics, statistics and Datetimes

- ▶ `math`: questo modulo mette a disposizione delle funzioni matematiche di base (`cos`, `sin`, `sqrt`, `pi`, ...).
- ▶ `random`: questo modulo fornisce degli strumenti per effettuare delle selezioni random.
- ▶ `statistics`: questo modulo fornisce strumenti statistici di base per analizzare una serie di dati.
- ▶ `datetime`: fornisce classi per gestire e manipolare date e ore con semplicità. Supporta anche calcoli matematici tra oggetti `datetime`.

Libreria joblib

- ▶ È una libreria di python che permette di salvare dati in rappresentazione binaria (pickle).
- ▶ json serializza in formato stringa mentre joblib in formato binario.
- ▶ Il modulo pickle è python dipendente, ovvero una volta che l'oggetto è stato serializzato in python, non è possibile usare un altro tipo di linguaggio per deserializzarlo.
- ▶ Per importare joblib: `from sklearn.externals import joblib`.

Numpy

- ▶ È la libreria principale e più utilizzata per il calcolo scientifico in python.
- ▶ Fornisce un oggetto array multidimensionale ad alte prestazioni e una serie di metodi con cui lavorare.
- ▶ Un ndarray è una griglia di valori, tutti dello stesso tipo, ed è indicizzato da una tripla di numeri interi positivi.
- ▶ Per importare la libreria: `import numpy`.
- ▶ Alcuni dei metodi principali: `.arange()`, `.random`, `.reshape()`, `.isnan()`, `.shape`, `.array()`, `.mean()`, `.std()`, `.min()`, `.median()`.

Pandas

- ▶ E' la principale libreria per la manipolazione e l'analisi dei dati in Python.
- ▶ Offre strutture dati e appositi metodi per la manipolazione di tabelle numeriche e serie temporali.
- ▶ L'oggetto principale fornito dalla libreria è l'oggetto indicizzato
- ▶ DataFrame per la manipolazione dei dati.
- ▶ Per importare la libreria: `import pandas`.
- ▶ Alcuni dei metodi principali: `.DataFrame()`, `.read_excel()`, `.read_csv()`, `.to_datetime()`, `.Series()`, `.concat()`, `.groupby()`, `.drop()`.

Esercizi

- ▶ Crea un package. Dentro di esso crea un modulo che definisce una funzione che genera una lista di 10 tuple, ognuna contenente due numeri casuali compresi tra 1 e 5.
- ▶ Il modulo deve essere importato nel main.py (che si troverà fuori dal package)

Esercizi

- ▶ Sempre nello stesso package crea un modulo dove viene importata la libreria random e viene definita una funzione CartaForbiceSasso().

Esercizi

- ▶ Crea un package, inserisci dentro un modulo contenente una classe studente, importarla nel main, inizializzare 2 istanze e stamparne i dati.
- ▶ Riprendere la classe studente vista nelle lezioni precedenti

Esercizi

- ▶ Crea un modulo contenente una classe rettangolo, una quadrato e una triangolo con i vari metodi per calcolare l'area
 - ▶ importare la libreria mathematics
- ▶ Aggiungere una funzione che calcoli l'ipotenusa di un triangolo rettangolo.

Lambda

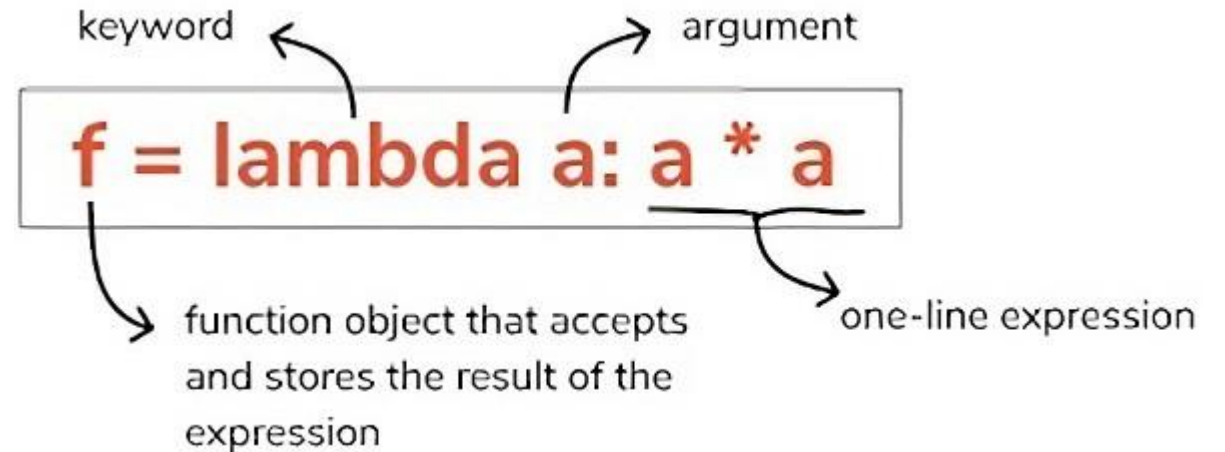
- ▶ Le lambda functions sono funzioni anonime che possono essere definite in una singola riga.
- ▶ Sono utili per scrivere codice breve e conciso.
- ▶ In Python, le funzioni lambda vengono create utilizzando la parola chiave 'lambda' seguita dagli argomenti della funzione e dall'espressione da valutare.



Lambda Functions

Lambda

- ▶ Keyword lambda
- ▶ Parametri in input
- ▶ Output
- ▶ La sintassi di base di una lambda è la seguente:
lambda parametri: espressione



Esempio

- ▶ #Definizione di una lambda per calcolare il quadrato di un numero

```
quadrato = lambda x: x ** 2
```

- ▶ #Utilizzo della lambda per calcolare il quadrato di un numero

```
numero = 5
```

```
risultato = quadrato(numero)
```

```
print("Il quadrato di", numero, "è:", risultato)
```

Lambda

- ▶ Le lambda sono spesso utilizzate con le funzioni built-in come:
 - ▶ `map()`,
 - ▶ `filter()`
 - ▶ `sorted()`
 - ▶ `reduce()`.
- ▶ Le funzioni built-in sono funzioni anonime che possono essere utilizzate direttamente nel codice senza doverle dichiarare esplicitamente con un nome.
- ▶ Per utilizzarle.... `from functool import nome_funzione`

Lambda

#restituisce una lista contenente tutti gli elementi dell'iterabile originale in ordine ascendente

```
numeri = [5, 2, 9, 1, 7]
```

```
ordina_rev = sorted(numeri, reverse=True) #Output: [9, 7, 5, 2, 1]
```

#applica cumulativamente una funzione a due argomenti agli elementi di una sequenza

```
numeri = [2, 3, 4, 5]
```

```
prod = reduce(lambda x, y: x * y, numeri) #Output: 120
```

#applica una funzione specificata a ogni elemento di un iterabile

```
numeri = [1, 2, 3, 4]
```

```
quadrati = list(map(lambda x: x ** 2, numeri)) #Output: [1, 4, 9, 16]
```

#costruisce un iteratore dagli elementi di un iterabile per i quali una funzione restituisce True.

```
numeri = [1, 2, 3, 4, 5]
```

```
pari = list(filter(lambda x: x % 2 == 0, numeri)) #Output: [2, 4]
```

Esercizi

- ▶ Creare una funzione lambda che prende due argomenti e restituisce la loro somma
- ▶ Utilizzare una funzione lambda con la funzione `reduce()` per trovare il prodotto di una lista di interi.
- ▶ Creare una lista di tuple, dove ogni tupla contiene un nome ed un'età. Utilizzare la funzione `sorted()` con una funzione lambda per ordinare la lista per età in ordine decrescente.
- ▶ Utilizzare una funzione lambda con la funzione `filter()` per filtrare le stringhe da una lista che hanno meno di 5 caratteri.

Esercizi

- ▶ Creare una lista di stringhe e utilizzare la funzione `map()` con una funzione lambda per restituire una nuova lista con ogni stringa in maiuscolo.
- ▶ Scrivere una funzione lambda che prende una stringa come argomento e restituisce la stringa invertita.
- ▶ Utilizzare una funzione lambda con la funzione `filter()` per filtrare i numeri pari da una lista di interi.
- ▶ Creare una lista di interi e utilizzare la funzione `map()` con una funzione lambda per elevare al quadrato ogni elemento della lista.

RegEx

- ▶ Le espressioni regolari, o RegEx, sono potenti strumenti per la corrispondenza dei modelli e la manipolazione del testo.
- ▶ In Python, le espressioni regolari possono essere utilizzate con il modulo "re".
- ▶ Questo modulo fornisce varie funzioni per la ricerca, la divisione e la sostituzione di stringhe in base a un dato schema.



RegEx

- ▶ La sintassi delle espressioni regolari comprende caratteri letterali e metacaratteri che rappresentano concetti come caratteri, gruppi di caratteri, quantificatori, assertive, etc.
 - ▶ . (che rappresenta qualsiasi carattere)
 - ▶ * (che indica zero o più occorrenze),
 - ▶ + (una o più occorrenze),
 - ▶ [] per specificare un insieme di caratteri),
 - ▶ etc.

Character	Description	Example
[]	Un insieme di personaggi	"[a-m]"
\	Segnala una sequenza speciale (può essere utilizzato anche per eseguire l'escape dei caratteri speciali)	"\d"
.	Qualsiasi carattere (tranne il carattere di nuova riga)	"he..o"
^	Inizia con	"^hello"
\$	Termina con	"planet\$"
*	Zero o più occorrenze	"he.*o"
+	Una o più occorrenze	"he.+o"
?	Zero o una occorrenza	"he.?o"
{}	Esattamente il numero di occorrenze specificato	"he.{2}o"
	Oppure	"falls stays"
()	Cattura e raggruppa	

RegEx

[Python RegEx](#)

Character	Description	Example
\A	Restituisce una corrispondenza se i caratteri specificati si trovano all'inizio del corda	"\AThe"
\b	Restituisce una corrispondenza in cui i caratteri specificati si trovano all'inizio o alla fine di una parola (la "r" all'inizio sta facendo in modo che la stringa sia essere trattato come una "stringa grezza")	r"\bain" r"ain\b"
\B	Restituisce una corrispondenza in cui sono presenti i caratteri specificati, ma NON all'inizio (o a la fine) di una parola (la "r" all'inizio fa in modo che la stringa viene trattato come una "stringa grezza")	r"\Bain" r"ain\B"
\d	Restituisce una corrispondenza in cui la stringa contiene cifre (numeri da 0 a 9)	"\d"
\D	Restituisce una corrispondenza in cui la stringa NON contiene cifre	"\D"
\s	Restituisce una corrispondenza in cui la stringa contiene uno spazio vuoto	"\s"
\S	Restituisce una corrispondenza in cui la stringa NON contiene uno spazio vuoto	"\S"
\w	Restituisce una corrispondenza in cui la stringa contiene caratteri alfanumerici (caratteri da dalla a alla Z, le cifre da 0 a 9 e il carattere di sottolineatura _)	"\w"
\W	Restituisce una corrispondenza in cui la stringa NON contiene caratteri alfanumerici	"\W"

RegEx

[Python RegEx](#)

RegEx

```
import re
```

```
testo = "La mela è rossa, la banana è gialla e l'arancia è arancione." #Stringa di esempio
```

```
regex parole_con_a = re.findall(r"\ba\w*", testo) #Trova tutte le parole che iniziano con la  
#lettera "a" nella stringa utilizzando il  
#pattern
```

```
print("Parole che iniziano con 'a':")
```

```
for parola in parole_con_a: print(parola) #Stampare le parole trovate
```

RegEx

- ▶ `regex parole_con_a = re.findall(r"\ba\w*", testo)`
 - ▶ Il metacarattere `\b` indica un limite di parola
 - ▶ `a` rappresenta la lettera "a"
 - ▶ `\w*` rappresenta zero o più caratteri alfanumerici che seguono la "a".

RegEx

- ▶ `re.findall(re, str)` → restituisce una lista contenente tutti i match trovati
- ▶ `re.match(re, str)` → Cerca la stringa per una corrispondenza e restituisce un «oggetto Match» se è presente una corrispondenza. usare `group()` o `start()` per prendere il primo elemento
- ▶ `re.sub(re, new_str, str)` → rimpiazza un testo trovato nella stringa con «new_str»
- ▶ [Python RegEx](#)

Esercizi

- ▶ Trovare la parola «ciao» in una stringa
- ▶ Trovare tutte le occorrenze di un numero in una stringa.
- ▶ Sostituire tutte le occorrenze di un carattere in una stringa.
- ▶ Trovare tutte le parole che iniziano con la lettera "p" in una stringa.
- ▶ Trovare tutte le parole che terminano con la lettera "o" in una stringa.
 - ▶ Hint : modulo re di python

Esercizi

- ▶ Trovare tutte le parole che contengono esattamente 4 lettere in una stringa.
- ▶ Trovare tutte le parole che contengono solo lettere minuscole in una stringa.
- ▶ Trovare tutte le parole che contengono solo lettere maiuscole in una stringa.
- ▶ Trovare tutte le occorrenze di una stringa che inizia con "http" o "https".
- ▶ Trovare tutte le occorrenze di una stringa che rappresentano un indirizzo email.
 - ▶ Hint : modulo re di python

Debugger

- ▶ PyCharm offre un potente debugger integrato per facilitare l'individuazione di eventuali errori nel codice Python.
- ▶ Si può avviare debugger facendo clic sul pulsante di debug o impostando dei punti di interruzione direttamente nel codice.
- ▶ Il debugger consente di controllare l'esecuzione del codice passo dopo passo e di esaminare le variabili in ogni punto del programma.
- ▶ "Step Over" per eseguire la riga corrente senza entrare nelle funzioni,
- ▶ "Step Into" per entrare nelle funzioni
- ▶ "Step Out" per uscire da una funzione corrente.

Debugger

```
1 usage
1  def calcola_media(lista):  lista: [10, 20, 30, 40, 50]
2      somma = 0  somma: 60
3      for numero in lista:  numero: 40
4          somma += numero
5      media = somma / len(lista)
6      return media
7
8
9  numeri = [10, 20, 30, 40, 50]
● risultato = calcola_media(numeri)
11 print("La media dei numeri è:", risultato)
```